IST 387: Introduction to Applied Data Science
Lab 2: Part 2 Notes

Notes Prepared by: Dr. Preeti Jagadev

Reference: Data Science for Business with R, written by Jeffrey S. Saltz and Jeffrey M. Stanton

Contents of the Notes:

- Introduction to Data Munging
- Reading a CSV Text File
- Removing Rows and Columns
- Renaming Rows and Columns
- Cleaning up the Elements
- Sorting and Subsetting Dataframes
- TIDYVERSE: An Introduction and How to Install the Package
- Sorting and Subsetting Dataframes using Tidyverse
- Important R Functions

Preeti Jagadev

Introduction to Data Munging

Data Munging:

Is the process of turning a dataset with a bunch of junk in it into a nice clean dataset.

Significance of Data Munging:

A data file might have information that is useful for humans, but not for R. With large datasets it is not practical to just type data into R. We need to be able to read in a large dataset and use various tools in R to clean the dataset and get it ready to analyze. This kind of work is a big part of data science, and very important to get right.

Reading a CSV Text File

In this chapter, we will explore how to read in a dataset that is stored as a comma-delimited text file (known as a CSV file- which stands for comma-separated values) that needs to be cleaned up.

There are many formats that we might have to be able to process to get data into R, but for now we will focus on a very common human-readable file format. Our first real dataset will be U.S. census data. The U.S. Census Bureau has stored population data in many locations on its website, with many interesting data sets to explore. We will use one of the simpler datasets available from the list shown at this website www2.census.gov/programs-surveys/popest/tables/2010-2011/state/totals/.

Click on the CSV link for nst-est2011-01.csv, and download the CSV.

Now, lets create a R code to read this dataset:

```
library(readr)

urlToRead <- "https://www2.census.gov/programs-surveys/popest/tables/2010-2011/state/totals/nst-est2011-01.csv"

testFrame <- read_csv(url(urlToRead ))
```

- The first command makes the readr package available. If you have not previously installed it, you will need to run **install.packages("readr")** first. The readr package provides a modern, straightforward way of reading in a CSV file and should be used in place of the older read.csv() function whenever possible.
- The second command just defines the location (on the web) of the file to load. Make sure that the URL is one continuous string with no spaces or line breaks.
- The third command reads the file, using the **read_csv ()** command from the **readr** package. Note, we also use the **url( ) function,** so R knows that the filename is a URL (as opposed to a local file on the computer).

Next, let's look at what we got. We can use the **str ()** function to create a summary of the structure of testFrame:

Preeti Jagadev

```
> str(testFrame)
```

The output is as follows:

```
spc_tbl_ [66 × 10] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ table with row headers in column A and column headers in rows 3 through 4. (leading dots indicate sub-part
s): chr [1:66] "Table 1. Annual Estimates of the Population for the United States, Regions, States, and Puerto
Rico:  April 1, "| __truncated__ "Geographic Area" NA "United States" ...
 $ ...2
: chr [1:66] NA "April 1, 2010" "Census" "308,745,538" ...
 $ ...3
: chr [1:66] NA NA "Estimates Base" "308,745,538" ...
 $ ...4
: chr [1:66] NA "Population Estimates (as of July 1)" "2010" "309,330,219" ...
 $ ...5
: num [1:66] NA NA 2.01e+03 3.12e+08 5.55e+07 ...
 $ ...6
: logi [1:66] NA NA NA NA NA NA ...
 $ ...7
: logi [1:66] NA NA NA NA NA NA ...
 $ ...8
: logi [1:66] NA NA NA NA NA NA ...
 $ ...9
: logi [1:66] NA NA NA NA NA NA ...
 $ ...10
: logi [1:66] NA NA NA NA NA NA ...
 - attr(*, "spec")=
  .. cols(
```

```
  ..   `table with row headers in column A and column headers in rows 3 through 4. (leading dots indicate sub-p
arts)` = col_character(),
  ..   ...2 = col_character(),
  ..   ...3 = col_character(),
  ..   ...4 = col_character(),
  ..   ...5 = col_number(),
  ..   ...6 = col_logical(),
  ..   ...7 = col_logical(),
  ..   ...8 = col_logical(),
  ..   ...9 = col_logical(),
  ..   ...10 = col_logical()
  .. )
 - attr(*, "problems")=<externalptr>
```

Now you understand what data scientists mean by junk in their data.

The overall structure is 66 observations of 10 variables, signifying that the spreadsheet contained 66 rows and 10 columns of data. Note "**spec_tbl_df", "tbl_df", "tbl",** and **"data.frame"** at the beginning of the output. The first three of these refer to an especially useful type of a dataframe called a **"tibble",** which we will be using quite often.

Removing Rows and Columns

Sometimes the data under consideration is a strange mix of character, numeric, and logical data. Clearly, we have some work to do if we are to make use of this data. This is common for data scientists, where sometimes the data is available, but needs to be cleaned up before it can be used. So, let's get data munging!

First, lets review one way to access a list, a vector, or a dataframe. In R, square brackets allow indexing into a list, vector, or dataframe. For example: myList[3] would give us the third element of myList. Keeping in mind that a dataframe is a rectangular, two-dimensional structure, we can address any element of a dataframe with both a row and column designator: myFrame[4,1] would give the fourth row and the first column. A shorthand for taking the whole column of a dataframe

is to leave the row index empty: myFrame[, 6] would give every row in the sixth column. Likewise, a shorthand for taking a whole row of a dataframe is to leave the column index empty: myFrame[10,] would give every column in the tenth row. We can also supply a list of rows instead of just one row, like this: myFrame[c(1,3,5),] would return rows 1,3,5 (including the data for all columns, because we left the column index blank).

Using this knowledge, we will use an easy trick to get rid of the stuff we don't need.

- **Example:**

Consider we have imported the following csv data frame, named **example.csv** (next page).

By default `read_csv` command reads the first row of CSV as column names. Thus we can get rid of the remaining top 7 rows, because they are irrelevant to our dataframe. The command is as follows:

```
install.packages("readr")
library(readr)
testFrame <- read_csv("example.csv")
testFrame
```

Table: CSV Input Dataframe

this file contains data for studying data munging										
the third row contains column headers and last 2 rows contain summary										
extra line										
extra line										
extra line										
extra line										
extra line										
extra line										
name	age	height	gender	hobby	marital sta	Ambition	Address	Education	Contact details	
a1	34	58	M	games						
a2	65	78	F	TV						
a3	72	28	F	TV						
a4	24	89	M	games						
a5	56	77	F	skating						
a6	34	58	M	games						
a7	65	78	F	TV						
a8	72	28	F	TV						
a9	24	89	M	games						
a10	56	77	F	skating						
a11	34	58	M	games						
a12	65	78	F	TV						
a13	72	28	F	TV						
a14	24	89	M	games						
a15	56	77	F	skating						
a16	34	58	M	games						
a17	65	78	F	TV						
a18	72	28	F	TV						
a19	24	89	M	games						
a20	56	77	F	skating						
a21	34	58	M	games						
a22	65	78	F	TV						
a23	72	28	F	TV						
a24	24	89	M	games						
a25	56	77	F	skating						
a26	34	58	M	games						
a27	65	78	F	TV						
a28	72	28	F	TV						
a29	24	89	M	games						
a30	56	77	F	skating						
a31	34	58	M	games						
a32	65	78	F	TV						
a33	72	28	F	TV						
a34	24	89	M	games						
a35	56	77	F	skating						
a36	34	58	M	games						
a37	65	78	F	TV						
a38	72	28	F	TV						
a40	56	77	F	skating						
a41	34	58	M	games						
a42	65	78	F	TV						
a43	72	28	F	TV						
a44	24	89	M	games						
a45	56	77	F	skating						
a46	34	77	M	games						
a47	65	58	F	TV						
a48	72	78	F	TV						
a49	24	28	M	games						
a50	56	89	F	skating						
a51	44	77	M	games						
a52	55	44	F	TV						
students of a university										
5 students of a university										
tail line										
tail line										
tail line										

Preeti Jagadev

The output dataframe is as follows:

```
> testFrame
# A tibble: 65 × 10
   this file contains data…¹ ...2  ...3  ...4  ...5  ...6  ...7  ...8  ...9
   <chr>                     <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr>
 1 the third row contains c… NA    NA    NA    NA    NA    NA    NA    NA
 2 extra line                NA    NA    NA    NA    NA    NA    NA    NA
 3 extra line                NA    NA    NA    NA    NA    NA    NA    NA
 4 extra line                NA    NA    NA    NA    NA    NA    NA    NA
 5 extra line                NA    NA    NA    NA    NA    NA    NA    NA
 6 extra line                NA    NA    NA    NA    NA    NA    NA    NA
 7 extra line                NA    NA    NA    NA    NA    NA    NA    NA
 8 name                      age   heig… gend… hobby mari… Ambi… Addr… Educ…
 9 a1                        34    58    M     games NA    NA    NA    NA
10 a2                        65    78    F     TV    NA    NA    NA    NA
# ℹ 55 more rows
# ℹ abbreviated name: ¹`this file contains data for studying data munging`
# ℹ 1 more variable: ...10 <chr>
# ℹ Use `print(n = ...)` to see more rows
```

As observed in the output above, we need to get rid of the rows 1:7. Thus we use the following command:

```
testFrame <- testFrame[-1:-7,]
testFrame
```

The minus sign used inside the square brackets refers to the index of rows that should be eliminated from the dataframe. Thus, the notation -1: -7 gets rid of the first seven rows. We also leave the column designator empty so that we can keep all columns for now. We assign the result back to the same data object, thereby replacing the original with our new, smaller cleaner version. The output is as follows:

```
> testFrame
# A tibble: 58 × 10
   this file contains data…¹ ...2  ...3  ...4  ...5  ...6  ...7  ...8  ...9
   <chr>                     <chr> <chr> <chr> <chr> <chr> <chr> <chr> <chr>
 1 name                      age   heig… gend… hobby mari… Ambi… Addr… Educ…
 2 a1                        34    58    M     games NA    NA    NA    NA
 3 a2                        65    78    F     TV    NA    NA    NA    NA
 4 a3                        72    28    F     TV    NA    NA    NA    NA
 5 a4                        24    89    M     games NA    NA    NA    NA
 6 a5                        56    77    F     skat… NA    NA    NA    NA
 7 a6                        34    58    M     games NA    NA    NA    NA
 8 a7                        65    78    F     TV    NA    NA    NA    NA
 9 a8                        72    28    F     TV    NA    NA    NA    NA
10 a9                        24    89    M     games NA    NA    NA    NA
# ℹ 48 more rows
# ℹ abbreviated name: ¹`this file contains data for studying data munging`
# ℹ 1 more variable: ...10 <chr>
# ℹ Use `print(n = ...)` to see more rows
```

Next, we can see that of the 10 variables we get from read_csv(), only the first five are useful to us (the last five seem to be blank). We can use the following command to keep the first five columns of the dataframe:

```
testFrame <- testFrame[,1:5]
testFrame
```

Next, we can use the **tail()** function to show us the last five rows.

```
tail(testFrame,5)
```

The output is as follows:

```
# A tibble: 5 × 5
  `this file contains data for studying data munging` ...2  ...3  ...4  ...5
  <chr>                                               <chr> <chr> <chr> <chr>
1 students of a university                            NA    NA    NA    NA
2 5 students of a university                          NA    NA    NA    NA
3 tail line                                           NA    NA    NA    NA
4 tail line                                           NA    NA    NA    NA
5 tail line                                           NA    NA    NA    NA
```

These rows contain information that we don't need. Let's eliminate those rows like this:

```
testFrame <- testFrame[-54:-58,]
tail(testFrame,5)
```

The output is as follows:

```
# A tibble: 5 × 5
  `this file contains data for studying data munging` ...2  ...3  ...4  ...5
  <chr>                                               <chr> <chr> <chr> <chr>
1 a48                                                 72    78    F     TV
2 a49                                                 24    28    M     games
3 a50                                                 56    89    F     skating
4 a51                                                 44    77    M     games
5 a52                                                 55    44    F     TV
```

There is still one bad entry, row between a38 and a40 is fully empty. We can get rid of any rows that contain missing data, using the **complete.cases()** command. This command analyses each row of the dataframe and returns the value of **TRUE** for every row that has all its data:

```
casesvector <- complete.cases(testFrame)
casesvector
```

The output is as follows:

```
> casesvector
 [1]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[14]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[27]  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[40] FALSE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE  TRUE
[53]  TRUE
```

Here one row is empty, thus we get a FALSE in the output. We can eliminate that row as follows:

```
testFrame <- testFrame[complete.cases(testFrame), ]
```

Renaming Rows and Columns

Now we are ready to perform a couple of data transformations. Let's continue with the previous example. We start with the following command:

```
head(testFrame,5)
```

The output is as follows:

```
> head(testFrame,5)
# A tibble: 5 × 5
  `this file contains data for studying data munging` ...2  ...3   ...4   ...5
  <chr>                                               <chr> <chr>  <chr>  <chr>
1 name                                                age   height gender hobby
2 a1                                                  34    58     M      games
3 a2                                                  65    78     F      TV
4 a3                                                  72    28     F      TV
5 a4                                                  24    89     M      games
```

The first row (red outline box) is supposed to be the column name and is not the actual data. The column names (green box outline) need to be changed. Thus,

1. **Let's delete the first row (red box outline above):**

```
testFrame <- testFrame[-1, ]
testFrame
```

The output is as follows:

```
> testFrame
# A tibble: 51 × 5
   `this file contains data for studying data munging` ...2  ...3  ...4  ...5
   <chr>                                               <chr> <chr> <chr> <chr>
 1 a1                                                  34    58    M     games
 2 a2                                                  65    78    F     TV
 3 a3                                                  72    28    F     TV
 4 a4                                                  24    89    M     games
 5 a5                                                  56    77    F     skating
 6 a6                                                  34    58    M     games
 7 a7                                                  65    78    F     TV
 8 a8                                                  72    28    F     TV
 9 a9                                                  24    89    M     games
10 a10                                                 56    77    F     skating
# ℹ 41 more rows
```

2. **Let's check the column names (green box outline above) of the dataframe:**

```
colnames(testFrame)
```

The output is as follows:

```
> colnames(testFrame)
[1] "this file contains data for studying data munging" "...2"
[3] "...3"                                              "...4"
[5] "...5"
```

3. Let's update the column names in the dataframe:

We can also use **colnames** to update the column names in the dataframe. We do this by having the colnames function on the left side of the assignment statement. We set this up by creating a character vector called **cnames** and listing the column names we want using the **c()** command.

```
cnames <- c("Name", "Age", "Height", "Gender", "Hobby")
colnames(testFrame) <- cnames
colnames(testFrame)
```

The output is as follows:

```
> colnames(testFrame)
[1] "Name"   "Age"    "Height" "Gender" "Hobby"
```

Now, let's check the first five rows of the dataframe:

```
head(testFrame,5)
```

The output is as follows:

```
> head(testFrame,5)
# A tibble: 5 × 5
  Name  Age   Height Gender Hobby
  <chr> <chr> <chr>  <chr>  <chr>
1 a1    34    58     M      games
2 a2    65    78     F      TV
3 a3    72    28     F      TV
4 a4    24    89     M      games
5 a5    56    77     F      skating
```

Cleaning up the Elements

We can change the formats and data types as needed.

- **Example:** Consider the csv (**example2.csv**) given below:

Region	Census	Estimate	April.2010	April.2011
Ala.bama	4,779,736	4,779,735	4,785,401	4802740
Alaska.	710,231	710,231	714,146	722718
.Arizona	6,392,017	6,392,013	6,413,158	6482505
Arkansas.	2,915,918	2,915,921	2,921,588	2937979
California.	37,253,956	37,253,956	37,338,198	37691912
Colo.rado	5,029,196	5,029,196	5,047,692	5116796
Fl.orida.	18,801,310	18,801,311	18,838,613	19057542
Georgia.	9,687,653	9,687,660	9,712,157	9815210
New York.	19,378,102	19,378,104	19,395,206	19465197
North Carolina.	9,535,483	9,535,475	9,560,234	9656401
.Tennessee	6,346,105	6,346,110	6,357,436	6403353
Tex.as	25,145,561	25,145,561	25,253,466	25674781
Virginia.	8,001,024	8,001,030	8,023,953	8096604
Washington.	6,724,540	6,724,540	6,742,950	6830038

Preeti Jagadev

Let's read the csv using the following command:

```
install.packages("readr")
library(readr)
testFrame <- read_csv("example2.csv")
testFrame
```

The output is as follows:

```
Rows: 14 Columns: 5
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): Region
num (4): Census, Estimate, April.2010, April.2011

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
> testFrame
# A tibble: 14 × 5
   Region            Census Estimate April.2010 April.2011
   <chr>              <dbl>    <dbl>      <dbl>      <dbl>
 1 Ala.bama         4779736  4779735    4785401    4802740
 2 Alaska.           710231   710231     714146     722718
 3 .Arizona         6392017  6392013    6413158    6482505
 4 Arkansas.        2915918  2915921    2921588    2937979
 5 California.     37253956 37253956   37338198   37691912
 6 Colo.rado        5029196  5029196    5047692    5116796
 7 Fl.orida.       18801310 18801311   18838613   19057542
 8 Georgia.         9687653  9687660    9712157    9815210
 9 New York.       19378102 19378104   19395206   19465197
10 North Carolina.  9535483  9535475    9560234    9656401
11 .Tennessee       6346105  6346110    6357436    6403353
12 Tex.as          25145561 25145561   25253466   25674781
13 Virginia.        8001024  8001030    8023953    8096604
14 Washington.      6724540  6724540    6742950    6830038
```

We can remove the dots in front of the state names very easily with the **gsub( )** command, which replaces all occurrences of a pattern with a new string. The "**g**" in **gsub( )** stands for global substitution. We want all the dots to be removed, so the **gsub( )** function is perfect because it will repeat the substitution process for every row.

```
testFrame$Region <- gsub("\\.", "", as.character(testFrame$Region))
```

The two backslashes in the string expression above are called **escape characters** and they force the dot that follows to be treated as a literal dot. The output is as follows:

```
Rows: 14 Columns: 5
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
chr (1): Region
num (4): Census, Estimate, April.2010, April.2011

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
> testFrame$Region <- gsub("\\.", "", as.character(testFrame$Region))
> testFrame
# A tibble: 14 × 5
   Region           Census Estimate April.2010 April.2011
   <chr>             <dbl>    <dbl>      <dbl>      <dbl>
 1 Alabama         4779736  4779735    4785401    4802740
 2 Alaska           710231   710231     714146     722718
 3 Arizona         6392017  6392013    6413158    6482505
 4 Arkansas        2915918  2915921    2921588    2937979
 5 California     37253956 37253956   37338198   37691912
 6 Colorado        5029196  5029196    5047692    5116796
 7 Florida        18801310 18801311   18838613   19057542
 8 Georgia         9687653  9687660    9712157    9815210
 9 New York       19378102 19378104   19395206   19465197
10 North Carolina  9535483  9535475    9560234    9656401
11 Tennessee       6346105  6346110    6357436    6403353
12 Texas          25145561 25145561   25253466   25674781
13 Virginia        8001024  8001030    8023953    8096604
14 Washington      6724540  6724540    6742950    6830038
```

Sorting and Subsetting Dataframes

Let's continue with the example considered in the previous section.

We should write the command to show the states with the largest increase in population.

- **Step 1:** We can do this by first calculating the change in population:

```
testFrame$change <- testFrame$April.2011 - testFrame$April.2010
```

- **Step 2:** We now calculate the average change across all the states:

```
avgchange <- mean(testFrame$change)
```

- **Step 3:** Making a new dataframe that only has the states with an above average increase. This line of code will produce the data for the above average states:

```
testFrame[testFrame$change > avgchange, ]
```

The output is as follows:

```
# A tibble: 3 × 6
  Region       Census Estimate April.2010 April.2011 change
  <chr>         <dbl>    <dbl>      <dbl>      <dbl>  <dbl>
1 California 37253956 37253956   37338198   37691912 353714
2 Florida    18801310 18801311   18838613   19057542 218929
3 Texas      25145561 25145561   25253466   25674781 421315
```

We get all the columns because we left the column index blank. Let's store the dataset for later use:

```
growingStates <- testFrame[testFrame$change > avgchange, ]
```

Now we must sort. Although we can sort a vector easily with the **sort( )** command, sorting a dataframe requires a new function called **order( ).** If we want to sort a dataframe, what we provide is an ordered list that shows which rows should appear in which order. We tell **order( )** which variable we want to sort on, and it will give back a list of row indices in the order we requested. Putting it all together yields this command:

```
growingStates <- growingStates[order(growingStates$change), ]
```

Working our way from the inside to the outside of the expression above, we want to sort in the order of the **growingSates$change** column. The **order function** will spit out a list of row indices that reflects the change in the population of the states. Let's look at the first five rows of our output:

```
head(growingStates,5)
```

The output is as follows:

```
# A tibble: 3 × 6
  Region       Census Estimate April.2010 April.2011 change
  <chr>         <dbl>    <dbl>      <dbl>      <dbl>  <dbl>
1 Florida    18801310 18801311   18838613   19057542 218929
2 California 37253956 37253956   37338198   37691912 353714
3 Texas      25145561 25145561   25253466   25674781 421315
```

The output is showing the states in increasing order of the change column. We can redo the sort but tell R to sort largest to smallest. We tell R we want the largest population first by adding an additional parameter to the order function, stating that it should be in decreasing order, which is done as follows:

```
growingStates <- growingStates[order(growingStates$change, decreasing = TRUE), ]
head(growingStates, 5)
```

The output is as follows:

```
# A tibble: 3 × 6
  Region       Census Estimate April.2010 April.2011 change
  <chr>         <dbl>    <dbl>      <dbl>      <dbl>  <dbl>
1 Texas      25145561 25145561   25253466   25674781 421315
2 California 37253956 37253956   37338198   37691912 353714
3 Florida    18801310 18801311   18838613   19057542 218929
```

We can see Texas has the highest increase, followed by California. This is only for the states with above-average population changes.

TIDYVERSE: An Introduction and How to Install the Package

The "**tidyverse**" is a collection of R packages that helps reorganize and visualize data. It consists of many packages to help with data manipulation (e.g. **dplyr**), working with data types (e.g. **stringr** for strings) and data visualization (e.g., **ggplot2**).

Method 1

- To install the **tidyverse** package, look in the lower-right-hand pane of RStudio.
- There are several tabs there, and one of them is Packages. Click on this and you will get a list of every package that you already have available in your copy of R with checkmarks for the ones that are ready to use.
- It is unlikely that **tidyverse** is already on this list, so click on the button that says Install Packages.
- Type the beginning of the package name in the appropriate area, and RStudio will start to prompt you with matching choices. Finish typing "tidyverse" " or choose it from the list.
- There might be a checked box for Install Dependencies, and if so, leave this checked.
- Once you click the Install button in this dialog, you will see some commands running on the R console (the lower-left pane).

- Once the installation is complete, you will see tidyverse added to the list in the lower-right pane (assuming you have clicked the Packages tab).
- One last step is to click the checkbox next to it. This runs the library 0 function on the package, which prepares it for further use.

Method 2

Another way to install tidyverse is to go into an R session and type:

```
install.packages("tidyverse")
```

To load **tidyverse** into your current R session, type the following command:

```
library(tidyverse)
```

You will need to do the library command any time you clear your environment (computer memory where all your variables are stored and can be cleared via the environment tab in the RStudio).

Sorting and Subsetting Dataframes using Tidyverse

Let's consider the previous example again and explore how we would show the states that are above average with respect to their population change, using tidyverse functions:

```
testFrame <- mutate(testFrame, Change = (April.2011 - April.2010))
avgChange <- mean( pull(testFrame, Change))
growingStates <- filter(testFrame, Change > avgChange)
growingStates <- arrange(growingStates, desc(Change))
growingStates <- slice(growingStates, 1:5)
select(growingStates, Region, April.2011, Change)
```

The output is as follows:

```
# A tibble: 3 × 3
  Region     April.2011 Change
  <chr>           <dbl>  <dbl>
1 Texas        25674781 421315
2 California   37691912 353714
3 Florida      19057542 218929
```

In the above code:

- We first calculate the **Change** using **mutate( ).**
- Then, we **pull** the **Change column**, and use that vector to calculate the **mean** change in the population.
- After that we create a new dataframe called **growingStates,** where we **filter** (create a subset) where the only rows in the dataframe are those with **Change** greater than the **avgChange.**
- The **growingStates** dataframe is next sorted via **arrange,** in descending order of the **Change column.**
- The **slice** command slices the dataset down to the first five rows. Finally, we **select** the columns we want to display.

Now, let's add a cool **tidyverse** feature called "**pipes**" and see how it looks. The **pipe** operator (two percent signs with a greater than in the middle: %>%) passes data from one command to the next one, enabling us to chain the commands together without the need for intermediate variables. Below is the code for the same:

```
testFrame %>% mutate(Change = (April.2011 - April.2010)) %>%
  filter(Change > mean(Change)) %>% arrange(desc(Change)) %>% slice(1:5) %>%
  select(Region, April.2011, Change)
```

The output is as follows:

```
# A tibble: 3 × 3
  Region     April.2011 Change
  <chr>           <dbl>  <dbl>
1 Texas        25674781 421315
2 California   37691912 353714
3 Florida      19057542 218929
```

We can also take the result of the final command and assign it to a new object with a right pointing assignment arrow as shown below:

```
testFrame %>% mutate(Change = (April.2011 - April.2010)) %>%
  filter(Change > mean(Change)) %>% arrange(desc(Change)) %>% slice(1:5) %>%
  select(Region, April.2011, Change) -> growingStates
```

Case Study: Reading, Cleaning, and Exploring a Survey Dataset

Let's explore a **small airline survey dataset**. First, we need to read in the small airline survey CSV datafile, using the **Import Dataset** within RStudio. Now that we have a survey dataset, let's explore it by first calculating the overall mean **likelihood to recommend (LTR),** as follows:

```
library(readr)
smallSurvey <- read_csv("smallSurvey.csv")
glimpse(smallSurvey)
```

You have used **str( )** to examine the data structures before. The tidyverse provides an alternative to **str( )** called **glimpse( ),** which we use above. The output is as follows:

```
> glimpse(smallSurvey)
Rows: 500
Columns: 5
$ Type.of.Travel <chr> NA, "Business travel", "Personal Travel", "Personal Travel", "Busin…
$ Airline.Status <chr> "Blue", "Blue", "Blue", "Blue", "Blue", "Gold", "Silver", "Blue", "…
$ Gender         <chr> "Female", "Female", "Male", "Female", "Female", "Female", "Female",…
$ Age            <dbl> 22, 45, 39, 64, 26, 50, 36, 54, 31, 50, 57, 50, 61, 33, 64, 17, 80,…
$ LTR            <dbl> 7, NA, 8, 6, 9, 7, 10, 4, 4, 8, 3, 10, 6, 9, 3, 4, 3, 8, 1, 2, 10, …
```

We can observe some NAs in the output. To clean this dataset, we will remove any survey without an LTR response, because without an LTR response, that survey will not clarify drivers for improving LTR. However, we will leave NAs in the other columns since it's not clear what to do (e.g. if someone does not provide their age):

```
#Remove NAs from the LTR column
smallSurvey <- filter(smallSurvey, !is.na(LTR))

#What is the average Likelihood.to.recommend?
mean(smallSurvey$LTR)
```

```
#The mean Likelihood.to.recommend by Type.ofTravel
smallSurvey %>% group_by(Type.of.Travel)%>% summarize(mean=mean(LTR), count=n())
```

The output is as follows:

```
# A tibble: 4 × 3
  Type.of.Travel   mean count
  <chr>           <dbl> <int>
1 Business travel  8.14   303
2 Mileage tickets  8.05    39
3 Personal Travel  5.46   156
4 NA               7        1
```

Note that when running the **summarize(),** we use **n( )** to show the number of instances in that specific group.

In the code below, we analyze LTR by age:

```
smallSurvey %>% mutate(Age, Age=round(Age, digits=-1)) %>%
  group_by(Age) %>%  summarize(mean= mean(LTR), count=n()) %>%
     arrange(desc(mean))
```

The output is as follows:

```
# A tibble: 7 × 3
    Age  mean count
  <dbl> <dbl> <int>
1    50  8.12    85
2    30  8.06    79
3    40  7.82   120
4    20  6.75    57
5    60  6.67    79
6    70  6       36
7    80  5.67    43
```

Rounding to a negative number of digits means rounding to a power of 10, so for example **round(x, digits = -1)** round to the nearest ten.

Important R Functions

- **arrange( )**: Returns the sorted observations in a dataset in ascending or descending order based on one of its variables.*
- **c( ):** Concatenates/combines items into a vector.
- **colnames( ):** Gets/sets the column names for the dataframe.
- **desc( )**: Organizes a vector in descending order-useful with arrange().*
- **filter( ):** Returns a selected subset of rows in a dataframe, based on evaluating a condition.*
- **glimpse( ):** Shows basic information about a vector or dataframe.*
- **group_by( ):** Organizes data into groups.*
- **gsub( ):** Substitutes one string for another.
- **head( ):** Shows the first few observations in a vector or dataframe.
- **library( ):** Makes a package available for use.
- **mean( ):** Calculates the arithmetic average of a set of numbers.
- **mutate( ):** Returns a new dataframe with an updated or new column.*
- **order( ):** Returns the indices in the order of the vector supplied.
- **read csv( ):** Reads in a CSV file.*
- **select( ):** Returns a subset of columns within a dataframe.*
- **slice( ):** Returns a subset of row within a dataframe.*
- **summary( ):** Summarizes a vector or dataframe.
- **summarize( ):** Summarizes data with selected functions.*
- **tail( ):** Shows the last few observations in a vector or dataframe.
- **url( ):** Makes sure R knows that the file is a URL (not a local file).

*Indicates a tidyverse function.

Preeti Jagadev